ARCHITECTURE REVIEWS, TECHNICAL DECISION RECORDS, AND ENGINEERING GOVERNANCE

Senior Engineering Case Study

Author	Venkata Lakshmi Parimala Pasupuleti
Primary focus	Architecture decisions, design reviews, technical trade-offs, and follow-through
Systems represented	Affordable Housing, data pipelines, RabbitMQ, migrations, documents, LeaseI, AgentProbe
Leadership model	Technical ownership and cross-functional influence without formal governance authority
Document type	Senior engineering case study
Implementation boundary	Direct experience plus clearly labeled target practices
Operating principle	Architecture is complete only when production evidence confirms the decision

This document does not claim ownership of a formal enterprise architecture board. It captures the review, decision, documentation, and delivery practices demonstrated through systems I designed, owned, migrated, supported, or modernized.

Document Map

1. Executive Summary
2. Scope and Governance Boundary
3. Architecture as a Decision System
4. Decision Lifecycle
5. Framing the Problem
6. Quality Attributes and Constraints
7. Architecture Document Structure
8. Technical Decision Records
9. Preparing for Design Review
10. Conducting the Review
11. Trade-off Analysis
12. Data and Database Decisions
13. Asynchronous and Integration Decisions
14. Migration and Release Decisions
15. AI Architecture Governance
16. Security, Reliability, Performance, and Cost
17. Cross-Team Dependencies and Ownership
18. Decision Follow-Through
19. Metrics and Review Effectiveness
20. Failure Modes and Anti-Patterns
21. Modernization Roadmap
22. Reusable Templates and Checklists
23. Lessons Learned

1. Executive Summary

Architecture is the set of decisions that shapes system behavior, operating risk, and future change. A strong design is not defined by the number of technologies it uses. It is defined by how clearly it connects a business problem to boundaries, data, contracts, reliability, security, deployment, and ownership.

My engineering experience involved decisions with material production consequences: annual HUD data publication, stored-procedure logic, RabbitMQ processing, document schema migration, database rollout, S3 integration, configuration dependencies, and agentic AI workflows. These systems required more than implementation. They required explicit reasoning about alternatives, failure modes, validation, and how the design would be operated after release.

This document presents architecture review as a continuous engineering discipline. It begins with problem framing, creates explicit decision records, uses cross-functional review to challenge assumptions, and verifies the decision through implementation and production evidence.

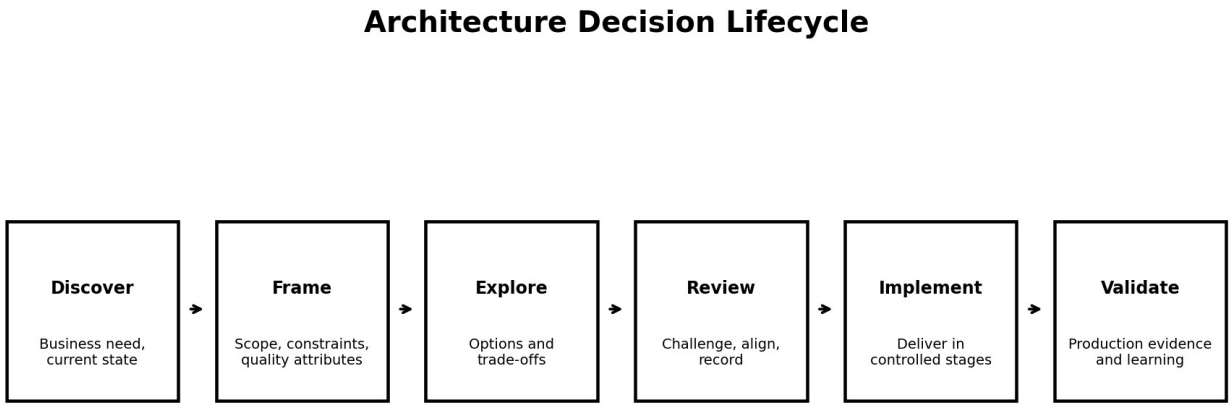


Figure 1. Architecture decisions move from discovery through review, implementation, and production validation.

2. Scope and Governance Boundary

My direct experience is technical ownership and influence rather than formal enterprise governance. I have made and reviewed decisions within modules and projects, coordinated with product, QA, database, platform, support, and other engineering teams, and participated in high-risk production releases. I did not own a company-wide architecture board, standards organization, or approval policy.

Area	Direct experience	Governance boundary
Application and data design	Schemas, stored procedures, ETL, APIs, workflows, validation	Within owned modules and project scope
Distributed processing	RabbitMQ producers, queues, workers, retries, idempotency concepts	Not ownership of the enterprise messaging platform

Area	Direct experience	Governance boundary
Migrations	Database, document, Git, and S3 integration planning and validation	Shared execution with database/platform/other teams
AI architecture	LeaseI and AgentProbe design, LangGraph, retrieval, tools, validation	Many production-scale controls remain target architecture
Technical standards	Review discipline, documentation, release and operational checklists	Not company-wide policy authority

3. Architecture as a Decision System

Architecture should make consequential decisions visible. If a design exists only in code or memory, the team cannot easily challenge its assumptions, onboard new engineers, or understand why a future change is risky.

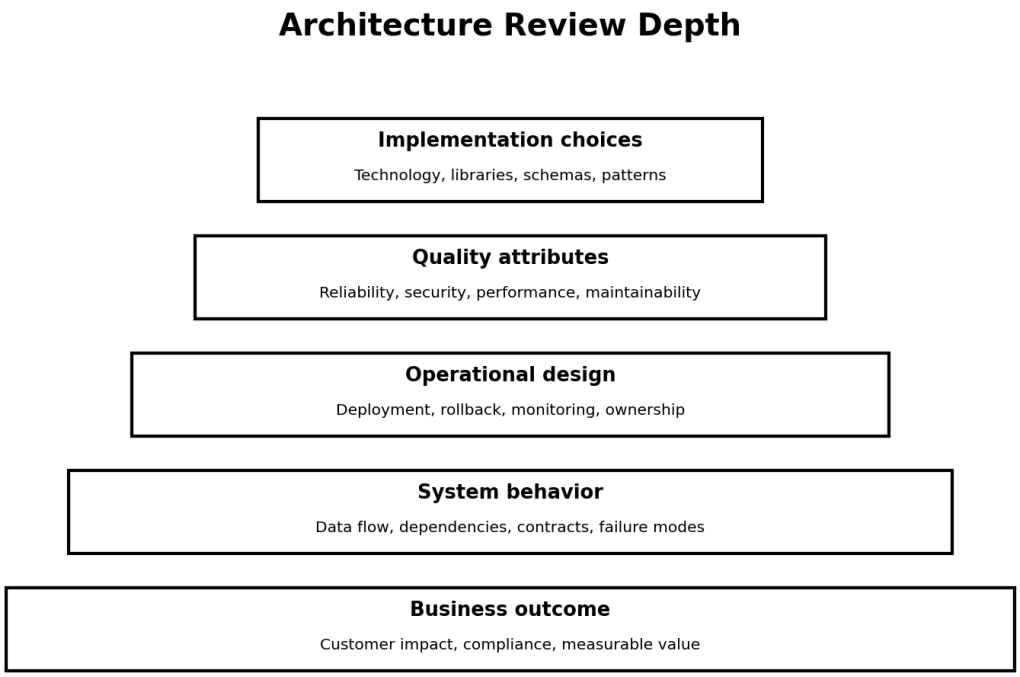


Figure 2. Strong architecture review begins with business outcomes and moves down to implementation choices.

- The business outcome defines what success means.
- System behavior explains how components, data, and workflows collaborate.
- Operational design defines deployment, recovery, monitoring, and ownership.
- Quality attributes define how reliable, secure, fast, scalable, and maintainable the system must be.
- Implementation choices select technologies and patterns after the higher-level needs are understood.

Starting with technology reverses this order and often produces unnecessary complexity. For example, RabbitMQ is valuable when work is long-running, retryable, independently scalable, and unsuitable for a blocking request. It is not automatically the right answer for every service interaction.

4. Decision Lifecycle

Stage	Primary output	Key questions
Discover	Current-state evidence and business problem	What is failing, expensive, slow, risky, or impossible today?
Frame	Scope, constraints, and quality attributes	What must be true, and what is explicitly outside scope?
Explore	Alternatives with trade-offs	What options exist, including keeping the current design?
Review	Challenged assumptions and aligned decision	Who is affected, and what evidence is missing?
Record	Decision, rationale, conditions, and owner	Why was this option selected, and when should it be revisited?
Implement	Incremental delivery and validation	How do we preserve compatibility and control risk?
Operate	Metrics, incidents, and customer outcomes	Did the decision work in production?

The lifecycle prevents a common failure: treating approval as the end of architecture work. Production behavior may reveal a hidden configuration dependency, a data distribution not represented in testing, or an operational burden larger than expected. The decision record should be updated or superseded when evidence changes.

5. Framing the Problem

Good architecture begins with a precise problem statement. A problem statement should describe the current behavior, affected users, business impact, evidence, and required outcome without embedding a preferred solution.

Weak framing	Stronger framing
We need microservices.	Annual processing blocks user requests, fails as one large unit, and cannot scale workers independently.
We need AI.	Support investigations require knowledge from product settings, regulations, historical tickets, and live data, causing repeated manual work.
The database is slow.	The annual transformation exceeds the release window because specific joins and expansion steps read substantially more data than they return.

Weak framing	Stronger framing
We should move to S3.	Affordable Housing documents need durable object storage while preserving metadata, ownership, retrieval behavior, and module compatibility.

- Identify the customer or operational workflow.
- Quantify scale, latency, frequency, and failure impact where evidence exists.
- Separate symptoms from root causes.
- Name the source of truth and required historical behavior.
- List explicit non-goals to prevent uncontrolled scope growth.

6. Quality Attributes and Constraints

Functional requirements describe what a system does. Quality attributes describe how well and how safely it must do it. They often determine the architecture more strongly than the happy-path feature list.

Attribute	Example in my systems	Architectural implication
Correctness	Eligibility and income limits must use the right rule and effective date	Deterministic logic, versioning, validation, audit evidence
Reliability	Annual imports and queued rent work must recover from failure	Workflow states, checkpoints, retries, idempotency, rollback
Performance	Large transformations must complete inside an operational window	Execution plans, indexing, batching, staged publication
Maintainability	Business rules and document mappings evolve across years	Clear boundaries, tests, documentation, controlled versioning
Security	Resident and customer data must remain authorized and isolated	Tenant scoping, least privilege, read-only diagnostics
Explainability	Support must understand why a result occurred	Decision records, rule traces, settings, evidence, historical data
Cost	AI and infrastructure should be proportionate to value and scale	Model routing, caching, staged infrastructure adoption

Constraints include existing schemas, team knowledge, annual deadlines, shared platforms, external APIs, customer compatibility, and the cost of migration. Ignoring constraints does not make a design ambitious; it makes it incomplete.

7. Architecture Document Structure

A practical architecture document should be short enough to review and detailed enough to expose risk. The level of detail should match the decision consequence.

Section	Purpose
Context	Business problem, users, current state, evidence
Goals and non-goals	Define success and control scope
Requirements	Functional behavior and quality attributes
Current architecture	Show what exists and where the problem occurs
Proposed architecture	Components, data flow, contracts, and states
Alternatives	Options considered and why they were not selected
Failure modes	How the system can fail and how failure is detected
Data and security	Source of truth, ownership, history, access, retention
Deployment and rollback	Sequence, compatibility, validation, recovery
Operations	Metrics, alerts, runbooks, and ownership
Open questions	Unknowns requiring experiments or decisions

Diagrams should clarify boundaries and flow. They should not replace written assumptions, data contracts, or failure behavior. A clean box-and-arrow diagram can still hide the hardest part of the design.

8. Technical Decision Records

A technical decision record preserves a specific consequential choice. It is narrower than a full design document and should remain understandable after the original participants move on.

ADR field	Content
Title	The decision in a concise phrase
Status	Proposed, accepted, superseded, deprecated, or rejected
Context	Problem, constraints, and evidence
Decision	What the team will do
Alternatives	Other credible options considered
Consequences	Benefits, costs, risks, and operational obligations
Conditions	Assumptions or scale thresholds that support the choice

ADR field	Content
Owner and date	Responsible technical owner and decision timing
Follow-up	Experiments, migrations, metrics, and revisit trigger

Examples of decisions worth recording include storing live HUD values through tool calls instead of embeddings, using RabbitMQ for long-running work, keeping regulated decisions deterministic, preserving effective-dated data, and adopting a phased document migration rather than an immediate cutover.

9. Preparing for Design Review

Review quality depends heavily on preparation. A meeting should not be the first time reviewers see the problem or discover that essential evidence is missing.

- Choose reviewers based on affected responsibilities: product, QA, database, platform, security, support, and domain ownership.
- Send the design with enough time for asynchronous reading.
- Highlight the decisions requiring feedback rather than asking for general approval.
- Bring data: execution plans, volumes, failure history, support impact, or benchmark results.
- List unresolved questions and assumptions explicitly.
- Prepare a small prototype or experiment when the main risk is technical uncertainty.
- Separate implementation details that can change later from architectural commitments that are expensive to reverse.

For the LeaseI target architecture, the highest-value review questions are not which agent names to use. They are how authorization is enforced across retrieval, which data is authoritative, how evidence is validated, how confidence is measured, and where humans remain accountable.

10. Conducting the Review

The purpose of review is to improve the decision, not to defend the author. The author should explain the problem, decision, major alternatives, and highest risks, then invite focused challenge.

Review behavior	Why it matters
Start with problem and goals	Prevents technology-first discussion
Confirm scope and non-goals	Avoids reviewing multiple projects at once
Examine assumptions	Most architecture failures begin with an untested assumption
Walk critical flows	Shows data, state, contracts, and failure behavior
Review operations	Ensures the system can be deployed, monitored, and recovered
Record decisions and actions	Prevents the meeting from becoming temporary conversation

Review behavior	Why it matters
Name dissent and conditions	Allows honest disagreement without ambiguous approval

Reviewers should distinguish blocking risks from optional improvements. Treating every preference as a blocker increases lead time and weakens trust. Conversely, approving a design without examining rollback, security, or source-of-truth behavior creates false confidence.

11. Trade-off Analysis

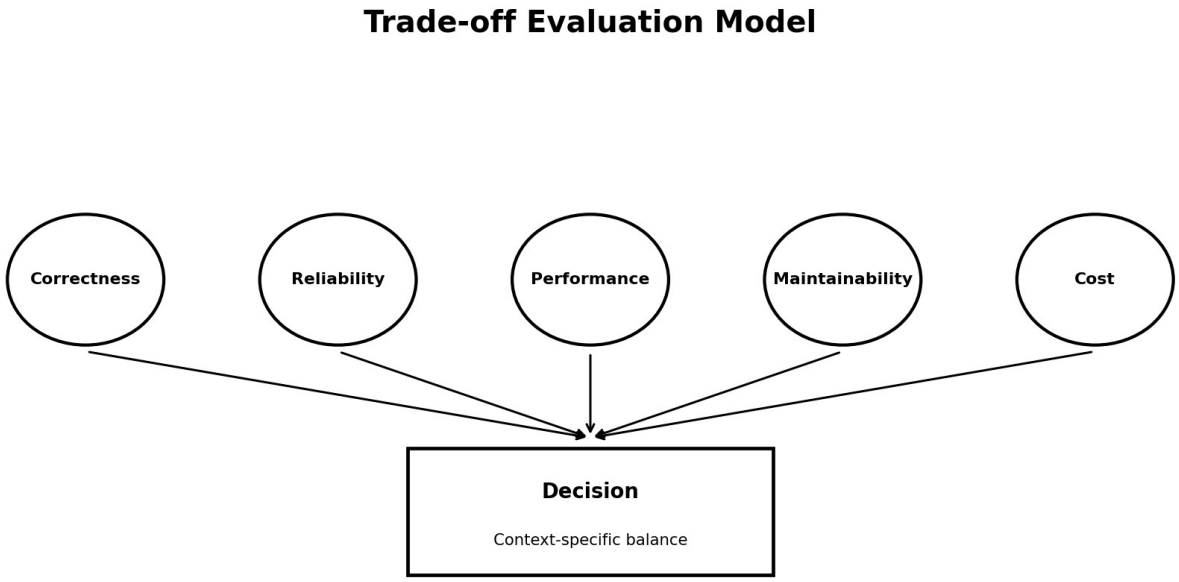


Figure 3. Architecture chooses a context-specific balance among competing quality attributes.

There is rarely one best architecture independent of context. The decision must state which qualities are prioritized and which costs are accepted.

Decision	Benefit	Accepted cost or risk
Stored procedures for heavy set-based transformations	Data-local execution and strong performance	Business logic can be harder to test and maintain across layers
RabbitMQ for annual gross-rent processing	Decoupling, retry, worker scaling, fault isolation	More operational state and eventual completion model
SQLite for current LeaseI build	Low operational overhead and fast iteration	Limited concurrency and production-scale capability
Read-only LeaseI tools	Reduced blast radius and safer adoption	Human must perform corrections separately

Decision	Benefit	Accepted cost or risk
Phased migration	Lower release risk and easier comparison	Longer coexistence and compatibility burden
RAG for changing knowledge	Freshness and source grounding	Retrieval quality and index governance become critical

Trade-offs should use evidence when possible. If a design claims better performance, test representative data. If it claims lower operational burden, identify the new components and on-call responsibilities. If it claims better AI quality, compare it against a versioned benchmark.

12. Data and Database Decisions

Database architecture decisions must consider source of truth, effective dates, cardinality, access patterns, transactions, migration, and operational ownership.

- Define business keys before selecting indexes.
- Preserve historical correctness through effective dating or versioned records.
- Use staging to isolate untrusted or incomplete source data from published data.
- Choose transaction boundaries according to business atomicity and recovery needs.
- Review execution plans and representative data distribution.
- Separate publication from transformation when partial visibility would be unsafe.
- Plan schema compatibility and data backfill independently when possible.

In the income-limit system, source records expanded into a much larger domain dataset. That transformation justified database-focused review of joins, indexes, stored procedures, reconciliation, and publication state rather than treating it as a simple file upload.

13. Asynchronous and Integration Decisions

The central question for asynchronous architecture is whether the initiating request must wait for completion. If work is long-running, independently retryable, or bursty, a queue can protect the user request and allow worker scaling.

Review question	Reason
What is the unit of work?	Determines message size, idempotency, and parallelism
Where is the source of truth?	Avoids using the message as an uncontrolled database
When is the message acknowledged?	Defines duplicate and loss behavior
Which failures are retryable?	Prevents infinite retries and hides less work
How is duplicate delivery handled?	At-least-once delivery requires idempotent effects
How is backlog measured?	Queue age and business delay drive capacity decisions
How is the workflow observed and recovered?	Asynchronous failure is separated from the original request

For service integrations, review should cover authentication, authorization, timeouts, retries, schema validation, versioning, and how the system behaves when the dependency is unavailable or returns stale data.

14. Migration and Release Decisions

Migrations are architecture because they define how the system moves between states. A target design without a safe transition design is incomplete.

Migration concern	Required decision
Compatibility	Can old and new application versions operate during rollout?
Data movement	How are records or documents mapped, copied, and reconciled?
Cutover	What condition makes the new path authoritative?
Rollback	Which code, schema, data, queue, and external state must be restored?
Validation	Which technical and business checks prove success?
Ownership	Who decides go, hold, rollback, or escalate?
Evidence	What is preserved to reconstruct the release?

The document merge-field migration showed how years of templates and contracts create compatibility risk. The Version 2 database rollout showed why timestamps, dumps, and phased evidence are essential when teams disagree about production state.

15. AI Architecture Governance

AI architecture needs the same governance as traditional systems plus controls for probabilistic behavior, retrieval, tool execution, model changes, and sensitive traces.

AI decision	Governance requirement
Knowledge source	Authority, freshness, access metadata, and trust tier
Retrieval	Evaluation, tenant filtering, metadata, empty-result behavior
Tool	Allowlist, schema, read/write scope, timeout, provenance
Model	Provider, version, routing, cost, latency, data policy

AI decision	Governance requirement
Prompt/workflow	Versioning, tests, injection resistance, state contract
Output	Evidence validation, unsupported-claim handling, human escalation
Release	Benchmark comparison, regression gate, rollback path
Operations	Traces, quality metrics, safety signals, retention

LeaseI currently implements a simpler architecture than the target. Reviews and documents must preserve that distinction: TF-IDF rather than hybrid retrieval, hand-tuned confidence rather than calibrated signals, fixed graph routing rather than formal model-selected tools, and SQLite rather than a production data platform.

16. Security, Reliability, Performance, and Cost

Cross-cutting attributes should be reviewed as explicit scenarios rather than broad statements.

Attribute	Scenario-based review
Security	Can a user retrieve another tenant's ticket, document, or setting by changing an identifier?
Reliability	What happens after the worker commits data but fails before acknowledgement?
Performance	How does the annual transformation behave at the largest expected data volume?
Recovery	Can the team resume from a checkpoint without duplicating published records?
Observability	Which signal identifies that business data is wrong even if services are healthy?
Cost	How do model calls, storage, workers, or database resources grow with workload?

Quality attributes should create acceptance criteria and tests. Saying a design is scalable or secure is not enough. The review should identify the expected workload, threat, failure, or recovery scenario and how it will be demonstrated.

17. Cross-Team Dependencies and Ownership

Architecture often fails at organizational boundaries rather than inside one component. The hidden configuration setting that changed Affordable Housing eligibility is a strong example. The technical rule was correct, but the dependency and ownership contract were not visible enough.

- Name the owning team for every external service, setting, schema, queue, document contract, and source dataset.
- Define change notification and compatibility expectations.
- Record operational and escalation contacts.
- Identify which team validates the end-to-end business outcome.
- Include shared dependencies in release readiness and incident triage.
- Treat configuration and release notes as part of the architecture contract.

A design review should ask not only whether each team can deliver its component, but whether the complete workflow has an owner who can validate the customer outcome.

18. Decision Follow-Through

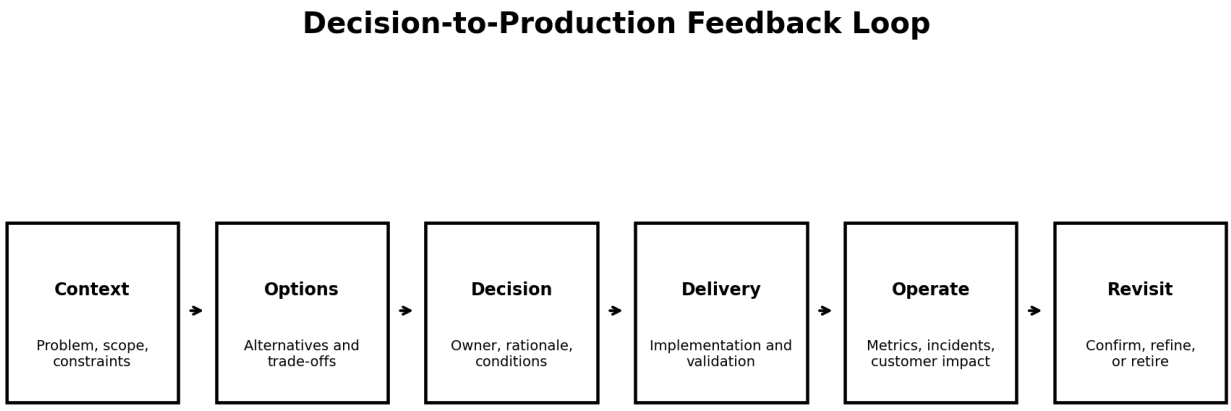


Figure 4. Architecture decisions should be confirmed, refined, or retired using production evidence.

A technical decision creates obligations. If RabbitMQ is selected, the team must operate queue metrics, retries, idempotency, DLQ handling, and replay. If RAG is selected, the team must operate ingestion, freshness, access filtering, retrieval evaluation, and citations.

Follow-through step	Evidence
Implementation review	Code and schemas match the approved boundaries and contracts
Pre-production validation	Quality attributes and failure scenarios are tested
Release validation	Checkpoints and business outcomes confirm the transition
Operational review	Metrics, alerts, runbooks, and ownership are active
Post-launch review	Real workload, incidents, cost, and user feedback are compared with assumptions

Follow-through step	Evidence
Decision update	ADR is confirmed, revised, superseded, or retired

This closes the gap between architecture as a document and architecture as real system behavior.

19. Metrics and Review Effectiveness

Architecture governance should improve outcomes rather than create meeting volume. Useful measures focus on decision quality, delivery clarity, and production learning.

Outcome	Possible measure
Decision clarity	Percentage of consequential changes with explicit owner, rationale, and revisit condition
Review efficiency	Time from review request to resolved decision, excluding uncontrolled waiting
Risk discovery	Material issues found before production versus during incidents
Delivery alignment	Implementation deviations identified before release
Operational readiness	Designs with validated monitoring, rollback, and runbooks
Decision durability	Frequency and reason for ADR supersession or emergency redesign
Knowledge reuse	New work referencing prior decisions and established patterns

Metrics require context. A higher number of superseded decisions can indicate poor judgment, but it can also indicate healthy learning in an uncertain domain. The goal is evidence-based improvement, not punishment for adaptation.

20. Failure Modes and Anti-Patterns

Technology-first design

Starting with a tool encourages the problem statement to be rewritten around the preferred solution.

Architecture by diagram

Boxes and arrows without state, contracts, failure, or ownership create a misleading sense of completeness.

Review as approval theater

A meeting where assumptions are not challenged adds delay without reducing risk.

Decision without alternatives

If no credible alternative is recorded, the rationale is difficult to evaluate later.

Design without migration

A target state that ignores compatibility, rollout, and rollback transfers risk to release time.

Governance without ownership

Standards and actions that have no accountable owner are rarely sustained.

Premature standardization

A pattern proven in one context may not justify company-wide mandate.

No production feedback

Architecture that is never revisited cannot learn from real scale and incidents.

21. Modernization Roadmap

Phase	Focus	Outcome
1. Standardize decision records	Lightweight ADR format and clear statuses	Rationale becomes durable and searchable
2. Define review triggers	Risk-based criteria for when review is required	Governance effort matches consequence
3. Improve review artifacts	Context, options, failure modes, migration, operations	Reviews examine the full system
4. Connect delivery	Implementation and release checklists reference decisions	Approved architecture survives execution
5. Add production feedback	Post-launch review and metrics	Assumptions become measurable
6. Build pattern catalog	Proven data, messaging, migration, and AI patterns	Teams reuse knowledge without copying blindly
7. Automate policy and evidence	Templates, CI checks, benchmark gates, migration validation	Repeated controls become consistent

22. Reusable Templates and Checklists

22.1 Architecture review checklist

- Is the business problem clear and supported by evidence?
- Are goals, non-goals, constraints, and scale explicit?
- Is the source of truth identified?
- Are credible alternatives and trade-offs documented?
- Are data, contracts, state, and failure modes clear?
- Are security, reliability, performance, and cost expressed as scenarios?

- Is the migration, compatibility, validation, and rollback plan complete?
- Are monitoring, ownership, and runbooks defined?
- Are open questions assigned with dates or experiments?

22.2 Technical decision record

- Title and status
- Context and evidence
- Decision and scope
- Alternatives considered
- Consequences and risks
- Conditions and revisit trigger
- Owner, date, and follow-up actions

22.3 Post-launch decision review

- Compare actual load, latency, failure, and cost with assumptions.
- Review customer and operational outcomes.
- Identify implementation deviations and why they occurred.
- Assess monitoring, recovery, and ownership effectiveness.
- Confirm, revise, supersede, or retire the decision.
- Update patterns, tests, documentation, and training.

23. Lessons Learned

- Architecture quality begins with problem clarity, not technology selection.
- The best design is a context-specific trade-off, not a universal pattern.
- Migration and operations are part of architecture, not follow-up tasks.
- Production evidence should be used to confirm or revise decisions.
- Configuration and cross-team dependencies require the same visibility as code contracts.
- Decision records reduce repeated debate and help future engineers understand constraints.
- Review is valuable when it challenges assumptions and produces clear actions, not when it only grants approval.
- AI architecture must preserve the distinction between implemented capabilities and target design.
- Clear ownership and scope boundaries strengthen technical credibility.

Closing Perspective

My strongest architecture lessons came from systems that had to survive annual change, production incidents, complex data dependencies, and cross-team delivery. A senior engineer is responsible not only for proposing a technically credible design, but also for making the decision understandable, executable, operable, and revisable. Architecture becomes trustworthy when its reasoning is recorded and its outcomes are verified in production.